THE MINIMALIST EDGE AI CHALLENGE
Software Track

Pruning, Quantizing, and Distilling Heavy Models for Lean, Green Edge Hardware

A One-Day Team Competition
Open to Undergraduate and Graduate Teams (2-4 Members)

1. Overview

The Minimalist Edge AI Challenge invites student teams to confront “AI Bloat” directly. Over the course of one day, teams take a state-of-the-art, resource-hungry model and transform it into a lean, efficient system capable of running on constrained, low-power hardware. The central question every team must answer with their submission is simple to state and hard to achieve: can you preserve intelligence while radically cutting the resource and energy footprint of a model?

1.1 Eligibility

- Open to all undergraduate and graduate students registered at a recognized university.
- Teams must consist of 2 to 4 members.
- Each participant may register with only one team.

1.2 Recommended Skill Set

- Proficiency in Python (PyTorch and/or TensorFlow) and/or C++/Rust.
- Experience with model optimization techniques such as pruning and quantization.
- Knowledge of Linux systems and resource profiling tools.
- An interest in sustainable and efficient engineering practices.

1.3 The Green Trifecta

Every design decision a team makes should be weighed against four goals, referred to together as the Green Trifecta (the fourth criterion, Energy Parsimony, is scored as part of this combined objective):

1. Model Accuracy: maintain high predictive performance and reliability while operating under strict resource constraints.
2. Model Compression: use pruning, quantization, and distillation to shrink the model footprint.
3. Resource Efficiency: minimize peak RAM and CPU utilization so the model runs smoothly on minimalist hardware.
4. Energy Parsimony: achieve the lowest possible Joules-per-inference to maximize battery life and sustainability.

1.4 How It Works

- Physical hardware is not needed. All work happens in a simulated or containerized environment, using free, publicly available tools including the Qualcomm AI Hub profilers.
- The target reference device is a Qualcomm Arduino UNO Q, a low-power, constrained edge board. Because this exact board may not always appear in every simulator's device catalog, teams simulate against the closest available low-power embedded or mobile device profile and document that choice in their report.
- Teams start from a provided heavy, power-hungry baseline model and apply pruning, quantization, and knowledge distillation to make it as small and efficient as possible while keeping accuracy high.
- The final submission is a single compressed model file, accompanied by a short technical report and reproducible code. Judges run the submission through a standardized benchmarking rig that scores accuracy, file size, and theoretical compute efficiency.

> **Note:** All tools, datasets, and packages referenced in this packet are free and open source. No paid licenses, paid cloud credits, or proprietary datasets are required to complete this challenge.

2. Schedule

Total working time is approximately 8-9 hours.

Time	Activity
08:00 - 08:30	Check-in, team formation confirmation, environment sanity check
08:30 - 9:00	Kickoff briefing: rules, starter kit release
9:00 - 12:00	Competition
12:00 - 12:30	Lunch break
12:30 - 15:00	Competition continue
15:00 - 17:00	Final benchmarking, report writing, and submission (Sections 8-9), Judging
17:00 - 18:00	Awards & Wrap up

3. Environment Setup

All software used in this challenge is free and cross-platform. Python 3.10 or later is required. Instructions are provided for macOS, Windows, and Linux. Pick the section for your operating system, then continue with the shared package installation step.

3.1 macOS

Install Python using Homebrew (free package manager for macOS). If Homebrew is not yet installed, get it from https://brew.sh.

```
brew install python@3.11
python3.11 -m venv edge-ai-env
source edge-ai-env/bin/activate
```

> **Note:** On Apple Silicon (M1/M2/M3/M4) Macs, the same commands work unmodified. PyTorch ships native ARM64 wheels.

3.2 Windows

Install Python from the Microsoft Store, or via winget in PowerShell, or from https://www.python.org/downloads/windows/. Using winget:

```
winget install Python.Python.3.11
py -3.11 -m venv edge-ai-env
edge-ai-env\Scripts\Activate.ps1
```

> **Note:** If PowerShell blocks the activation script, run PowerShell as Administrator once and execute: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

3.3 Linux (Debian/Ubuntu-based)

```
sudo apt update
sudo apt install -y python3.11 python3.11-venv python3-pip
python3.11 -m venv edge-ai-env
source edge-ai-env/bin/activate
```

> **Note:** For Fedora/RHEL-based systems, replace the apt commands with: sudo dnf install -y python3.11 python3-pip

3.4 Shared Package Installation

With the virtual environment activated (all three operating systems), initialize your workspace and install the required packages. Note that on macOS the standard PyPI wheel already targets CPU, so the extra index URL is only needed on Windows and Linux to force the lightweight CPU-only build (and avoid a multi-gigabyte CUDA download that is not needed for this challenge).

Windows and Linux

```
python -m pip install --upgrade pip
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
pip install onnx onnxruntime thop codecarbon psutil qai-hub matplotlib
```

macOS

```
python -m pip install --upgrade pip
pip install torch torchvision torchaudio
pip install onnx onnxruntime thop codecarbon psutil qai-hub matplotlib
```

3.5 Package Reference

Package	Purpose	License / Source
torch, torchvision	Model definition, training, pruning, quantization	BSD, pytorch.org
onnx, onnxruntime	Model export and CPU inference for benchmarking	MIT/Apache-2.0, onnxruntime.ai
thop	FLOPs and parameter counting	MIT, PyPI
codecarbon	Energy and CO2 estimation of local compute	MIT, codecarbon.io
psutil	CPU and RAM profiling	BSD, PyPI
qai-hub	Qualcomm AI Hub client for device simulation and profiling	Free tier, aihub.qualcomm.com
matplotlib	Optional: plotting results for the team report	PSF/BSD-style, matplotlib.org

3.6 Qualcomm AI Hub Account (Free)

Create a free account at https://aihub.qualcomm.com. After signing in, open your account settings to generate an API token, then configure the local client:

```
qai-hub configure --api_token YOUR_TOKEN_HERE
```

This step is identical on macOS, Windows, and Linux once the virtual environment is active.

9. Submission Guidelines

Each team submits a single folder or archive containing the following items:

- Final compressed model file, preferably the quantized ONNX file (for example student_model_int8.onnx).
- metrics.json, produced as described in Section 8.
- A short technical report (1-2 pages, PDF or Word) summarizing the compression pipeline used, the chosen Qualcomm AI Hub device profile and rationale, and the final accuracy-versus-efficiency tradeoff.
- All training and export source code, organized so a judge can reproduce the pipeline from the baseline checkpoint.

Judges evaluate the submitted model on a standardized benchmarking rig that independently re-runs the accuracy, size, and compute efficiency measurements described in Sections 5 and 7. Submissions that cannot be reproduced from the provided code and checkpoint may be scored down under the Documentation and Reproducibility rubric criterion.

10. Evaluation Rubric

Total available score is 100 points, distributed across the Green Trifecta and two supporting criteria. Judges score every team using the same benchmarking rig and the same report checklist.

Criterion	Points	What Is Measured
Model Accuracy & Reliability	25	Task accuracy retained relative to the baseline heavy model, plus consistent behavior across the validation set
Model Compression	20	Reduction in parameter count and on-disk file size versus the baseline
Resource Efficiency	20	Peak RAM usage and CPU latency during inference, measured locally and via Qualcomm AI Hub
Energy Parsimony	20	Joules per inference, measured locally and cross-checked against the AI Hub device profile
Documentation & Reproducibility	10	Clarity of the technical report and whether judges can reproduce the pipeline from the provided code
Presentation & Engineering Judgment	5	Quality of the tradeoff analysis: does the team explain why it chose its particular mix of pruning, quantization, and distillation

10.1 Model Accuracy & Reliability (25 points)

Band	Points	Description
Excellent	22-25	Final model retains at least 97% of baseline accuracy on the held-out validation set, with stable predictions across all 10 classes.
Good	16-21	Final model retains 90-96% of baseline accuracy, with only minor weakness on one or two classes.
Fair	9-15	Final model retains 75-89% of baseline accuracy, or shows a clear weakness concentrated in a specific class.
Poor	0-8	Final model retains less than 75% of baseline accuracy, or accuracy figures cannot be reproduced by judges.

10.2 Model Compression (20 points)

Band	Points	Description
Excellent	18-20	At least a 10x reduction in both parameter count and on-disk file size relative to the baseline.
Good	13-17	A 5x to 10x reduction in parameter count and file size relative to the baseline.
Fair	7-12	A 2x to 5x reduction in parameter count and file size relative to the baseline.
Poor	0-6	Less than a 2x reduction, or compression claims cannot be verified from the submitted file.

10.3 Resource Efficiency (20 points)

Band	Points	Description
Excellent	18-20	Peak RAM and CPU latency both fall in the lowest quartile among all submissions, well suited to the target constrained device.
Good	13-17	Peak RAM and CPU latency are meaningfully below the baseline and competitive with most other submissions.
Fair	7-12	Some reduction versus baseline, but resource use remains high relative to other submissions.
Poor	0-6	Little or no improvement over the baseline, or the model fails to run within the benchmarking rig's resource limits.

10.4 Energy Parsimony (20 points)

Band	Points	Description
Excellent	18-20	Joules-per-inference in the lowest quartile among all submissions, with results corroborated by the Qualcomm AI Hub profile.

Good	13-17	Clear energy reduction versus baseline, with local and AI Hub measurements broadly consistent with each other.
Fair	7-12	Some energy reduction versus baseline, but with limited or missing AI Hub corroboration.
Poor	0-6	Little or no energy improvement, or energy figures are missing or not reproducible.

10.5 Documentation & Reproducibility (10 points)

Band	Points	Description
Excellent	9-10	Judges can rebuild the full pipeline from baseline checkpoint to final model using only the submitted code and report, without contacting the team.
Good	6-8	Pipeline is mostly reproducible with minor gaps or ambiguity in the report or code comments.
Fair	3-5	Pipeline is partially reproducible; key steps are described but not fully scripted.
Poor	0-2	Pipeline cannot be reproduced from the submission.

10.6 Presentation & Engineering Judgment (5 points)

Band	Points	Description
Excellent	5	Report clearly explains the tradeoffs between accuracy, compression, resource use, and energy, and justifies the final design choice with evidence from the team's own measurements.
Good	3-4	Report explains the pipeline and results, with a reasonable but less thorough discussion of tradeoffs.
Fair	1-2	Report lists results without much discussion of why particular choices were made.
Poor	0	Report is missing or does not describe the reasoning behind the submitted pipeline.

11. Appendix: Resource Links

PyTorch (free, open source): https://pytorch.org

TorchVision models and datasets: https://pytorch.org/vision/stable/index.html

ONNX: https://onnx.ai

ONNX Runtime and quantization tools: https://onnxruntime.ai

PyTorch pruning utilities: https://pytorch.org/tutorials/intermediate/pruning_tutorial.html

PyTorch quantization documentation: https://pytorch.org/docs/stable/quantization.html

THOP, FLOPs and parameter profiler: https://github.com/Lyken17/pytorch-OpCounter

CodeCarbon energy tracker: https://codecarbon.io

psutil system monitoring: https://psutil.readthedocs.io

Qualcomm AI Hub: https://aihub.qualcomm.com

11.1 Troubleshooting

- If pip install of torch is slow or times out, retry using the CPU-only index URL shown in Section 3.4, which avoids downloading unnecessary large GPU driver packages.
- If qai-hub commands report an authentication error, re-run qai-hub configure --api_token with a freshly generated token from the AI Hub account settings page.
- If a team's laptop has limited RAM, reduce batch_size in the DataLoader calls throughout this packet, for example from 32 to 8.
- On Windows, if activation scripts are blocked, use the Command Prompt activation path instead: edge-ai-env\Scripts\activate.bat